

Plan de Pruebas: Plataforma DataForge IA

1. Introducción

El presente documento describe el **Plan de Pruebas** para la plataforma **DataForge IA**, una solución web de análisis predictivo para PYMES. El objetivo principal es asegurar la calidad, fiabilidad, rendimiento y seguridad del software mediante la ejecución sistemática de diversas pruebas. Este plan detalla los tipos de pruebas a realizar, la estrategia de prueba, los recursos necesarios y un cronograma estimado, garantizando que el producto final cumpla con los requisitos funcionales y no funcionales definidos.

2. Tipos de Pruebas

Se aplicarán diferentes niveles y tipos de pruebas a lo largo del ciclo de desarrollo para asegurar una cobertura integral:

2.1. Pruebas Unitarias

- **Descripción:** Verifican la funcionalidad de los componentes de código más pequeños y aislados (funciones, métodos, clases). Son realizadas por los desarrolladores durante la fase de codificación.
- **Objetivo:** Asegurar que cada unidad de código funciona correctamente de forma independiente.
- **Herramientas:** `Pytest` (Python), `Jest` (JavaScript/TypeScript).

2.2. Pruebas de Integración

- **Descripción:** Verifican la interacción y comunicación entre diferentes módulos o servicios del sistema. Se centran en las interfaces y el flujo de datos entre componentes.
- **Objetivo:** Detectar errores en la comunicación y el intercambio de datos entre módulos.
- **Herramientas:** `Postman` / `Newman` para APIs, frameworks de pruebas de integración específicos para microservicios.

2.3. Pruebas Funcionales (Basadas en Historias de Usuario y Casos de Uso)

- **Descripción:** Verifican que las funcionalidades del sistema cumplen con los requisitos especificados en las Historias de Usuario y los Casos de Uso. Se realizan desde la perspectiva del usuario final.
- **Objetivo:** Confirmar que el sistema realiza las funciones esperadas correctamente.
- **Herramientas:** Cypress / Selenium (para UI), Postman (para API), pruebas manuales.

2.4. Pruebas de Sistema (End-to-End)

- **Descripción:** Verifican el sistema completo, de principio a fin, en un entorno que simula la producción. Incluyen la interacción con todos los componentes integrados (frontend, backend, base de datos, servicios externos).
- **Objetivo:** Validar el comportamiento general del sistema y la experiencia del usuario final.
- **Herramientas:** Cypress / Selenium .

2.5. Pruebas de Rendimiento

- **Descripción:** Evalúan la capacidad de respuesta, estabilidad, escalabilidad y uso de recursos del sistema bajo diferentes cargas de trabajo (pruebas de carga, estrés, volumen).
- **Objetivo:** Asegurar que el sistema cumple con los requisitos de rendimiento y puede manejar el volumen de usuarios y datos esperado.
- **Herramientas:** JMeter , Locust , k6 .

2.6. Pruebas de Seguridad

- **Descripción:** Identifican vulnerabilidades y debilidades en el sistema que podrían ser explotadas por atacantes. Incluyen pruebas de penetración, análisis de vulnerabilidades y revisión de código.
- **Objetivo:** Proteger el sistema contra accesos no autorizados, pérdida de datos y otros riesgos de seguridad.
- **Herramientas:** OWASP ZAP , Burp Suite , Snyk .

2.7. Pruebas de Usabilidad

- **Descripción:** Evalúan la facilidad de uso, la eficiencia y la satisfacción del usuario con la interfaz. Se realizan con usuarios reales o simulados.

- **Objetivo:** Asegurar que la plataforma es intuitiva y fácil de usar para el público objetivo (PYMES).
- **Técnicas:** Pruebas con usuarios, encuestas, análisis de tareas.

2.8. Pruebas de Regresión

- **Descripción:** Verifican que los cambios recientes en el código (nuevas funcionalidades, correcciones de errores) no han introducido nuevos defectos ni han afectado negativamente la funcionalidad existente.
- **Objetivo:** Mantener la estabilidad del sistema a medida que evoluciona.
- **Automatización:** Se automatizarán al máximo para ejecutarse en cada ciclo de CI/CD.

3. Estrategia de Pruebas

La estrategia de pruebas para DataForge IA se basará en un enfoque de **cambio a la izquierda (shift-left testing)**, integrando las actividades de prueba desde las primeras etapas del ciclo de desarrollo. Esto implica:

- **Pruebas Tempranas:** Los desarrolladores realizarán pruebas unitarias y de integración desde el inicio. Las pruebas serán parte integral de la definición de una funcionalidad.
- **Automatización:** Priorizar la automatización de pruebas en todos los niveles para permitir una ejecución rápida y repetible, especialmente en el pipeline de CI/CD.
- **Entornos de Prueba:** Se establecerán diferentes entornos de prueba para cada fase:
 - **Desarrollo:** Entornos locales para pruebas unitarias y de integración por parte de los desarrolladores.
 - **Integración/QA:** Entorno dedicado para pruebas de integración, funcionales y de sistema, gestionado por el equipo de QA.
 - **Pre-producción (Staging):** Entorno que replica la producción para pruebas de rendimiento, seguridad y regresión final antes del despliegue.
- **Gestión de Defectos:** Se utilizará un sistema de seguimiento de defectos (ej. Jira) para registrar, priorizar y gestionar los errores encontrados. Cada defecto tendrá un ciclo de vida claro (Abierto, En Progreso, Resuelto, Cerrado).
- **Criterios de Salida:** Un Sprint o una Release solo se considerará lista para el siguiente paso si se cumplen los siguientes criterios:
 - Todas las pruebas unitarias y de integración pasan.
 - Todas las pruebas funcionales críticas pasan.

- No hay defectos de prioridad Alta o Crítica abiertos.
- La cobertura de código alcanza el umbral definido (ej. 80%).
- Las pruebas de rendimiento y seguridad no revelan problemas significativos.

4. Recursos

Para la ejecución del plan de pruebas, se requerirán los siguientes recursos:

4.1. Equipo de Pruebas

- **QA Lead/Engineer:** Responsable de la planificación, diseño y supervisión de las actividades de prueba. Coordinará con el equipo de desarrollo y el Product Owner.
- **Desarrolladores:** Responsables de escribir y mantener las pruebas unitarias y de integración de su código.
- **Usuarios de Negocio (UAT):** Participarán en las pruebas de aceptación de usuario para validar que el sistema cumple con sus expectativas y necesidades de negocio.

4.2. Herramientas

Categoría de Herramienta	Herramientas Sugeridas	Propósito
Gestión de Pruebas	TestRail / Zephyr Scale	Planificación, ejecución y seguimiento de casos de prueba.
Automatización UI	Cypress / Selenium	Automatización de pruebas funcionales y E2E de la interfaz de usuario.
Automatización API	Postman / Newman	Automatización de pruebas de APIs REST.
Pruebas de Rendimiento	JMeter / Locust / k6	Simulación de carga de usuarios y medición de métricas de rendimiento.
Análisis de Seguridad	OWASP ZAP / Snyk	Identificación de vulnerabilidades de seguridad.
Gestión de Defectos	Jira / Trello	Registro, seguimiento y gestión del ciclo de vida de los defectos.
CI/CD	GitHub Actions / GitLab CI	Integración de pruebas automatizadas en el pipeline de desarrollo.

4.3. Entornos

- **Entorno de Desarrollo:** Máquinas locales de los desarrolladores.
- **Entorno de QA/Integración:** Servidor dedicado o contenedores en la nube para la integración continua y pruebas de QA.
- **Entorno de Pre-producción (Staging):** Réplica de producción para pruebas finales antes del despliegue.

5. Cronograma (Ejemplo Integrado con Sprints)

El cronograma de pruebas estará estrechamente integrado con el Plan de Desarrollo Ágil, ejecutándose de forma continua en cada Sprint. A continuación, se presenta un ejemplo de cómo las actividades de prueba se alinearán con los Sprints del MVP:

Sprint	Actividades de Prueba Clave	Duración Estimada
Sprint 1	- Pruebas unitarias (AuthService, DataManagementService)	

- Pruebas de integración (AuthService <-> UserMetadataDB)
- Revisión de seguridad de código (SAST) | 2 semanas | | **Sprint 2** | - Pruebas unitarias (DataManagementService, ObjectStorage)
- Pruebas de integración (DataManagementService <-> ObjectStorage <-> UserMetadataDB)
- Pruebas funcionales básicas de carga de datos | 2 semanas | | **Sprint 3** | - Pruebas unitarias (PredictionModelingService, VisualizationReportingService)
- Pruebas de integración (PredictionModelingService <-> MessageQueue <-> PredictionResultsDB)
- Pruebas funcionales de configuración de modelo y visualización
- Pruebas de rendimiento iniciales (API Gateway) | 2 semanas | | **Post-MVP (Ejemplo)** | - Pruebas de sistema (E2E) completas
- Pruebas de regresión automatizadas
- Pruebas de rendimiento y estrés exhaustivas
- Pruebas de seguridad (penetración) | Continuo / Sprints dedicados |

6. Conclusiones

El Plan de Pruebas para DataForge IA es un componente crítico para asegurar la entrega de un producto de software de alta calidad. Al adoptar un enfoque de pruebas tempranas y automatizadas, y al integrar diversas técnicas de prueba a lo largo del ciclo de desarrollo, se minimizan los riesgos y se maximiza la confianza en la plataforma. Este plan, junto con la colaboración entre los equipos de desarrollo y QA, garantizará que DataForge IA sea una solución robusta, fiable y segura para las PYMES.

7. Referencias

[1] ISTQB. (n.d.). *ISTQB Glossary of Testing Terms*. International Software Testing Qualifications Board. [2] Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley.

- Documentación Técnica: DataForge IA.

- Estándares de Desarrollo: DataForge IA.
- Estrategia de Control de Versiones: DataForge IA.